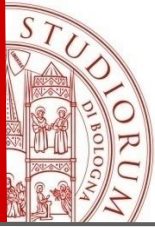


Sui numeri finiti utilizzando Python

Elena Loli Piccolomini

Calcolo Numerico 2025-26



Sistema floating point

Per avere informazioni sul sistema floating point utilizzato in Python:

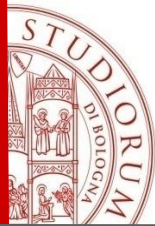
```
import sys  
sys.float_info
```

La distanza fra due numeri floating point può essere calcolata con la funzione *spacing* della libreria **numpy**. Per esempio se vogliamo calcolarla nel valore 10^9 :

```
import numpy as np
```

```
np.spacing(1e9)
```

Verificare che questo è vero (per esempio aggiungere un numero minore del gap a 10^9 e controllare il risultato).



Sistema floating point

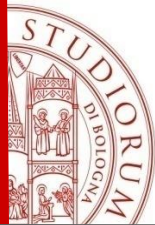
Ci sono poi dei casi speciali per esempio quando l'esponente è 0 oppure quando l'esponente è massimo, per rappresentare quelli che sono denominati *Not a Number (NaN)* oppure infiniti (*Inf* e *-Inf*).

Il massimo numero positivo floating point rappresentabile:

```
sys.float_info.max
```

I numeri maggiori del massimo provocano overflow e in Python sono Denominati *Inf*.

Esercizio: Verificare che, aggiungendo un numero piccolo (per esempio 2) al massimo si ottiene sempre il massimo mentre aggiungendo un numero Sufficientemente grande si ottiene inf.



Errori di rappresentazione

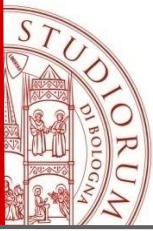
Nei numeri reali la differenza fra 4.9 e 4.845 è 0.005.

Utilizzando il sistema floating point di Python questo non è vero:

```
4.9 - 4.845
```

```
0.05500000000000000604
```

Esercizio. Verificare che 0.1+0.2+0.3 NON è uguale a 0.6 nel sistema Floating Point di Python.



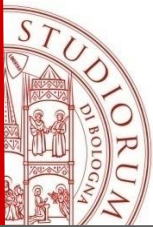
Propagazione degli errori

Nei numeri reali la differenza fra 4.9 4.845 è 0.005.
Utilizzando il sistema floating point di Python questo non è vero:

```
# If we only do once  
1 + 1/3 - 1/3
```

```
1.0
```

Esercizio. Ripetere l'esempio precedente che somma e sottrae 1/3 a 1 un
Numero N di volte. Testare con N=10,100,1000,10000,1000000.

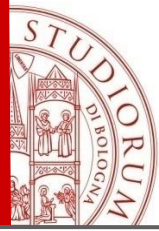


Propagazione degli errori

Esercizio. Scrivere una funzione python per calcolare il risultato di:

$$\left(1 + 1/n\right)^n$$

Con $n=1, 10, 100, \dots, 10^{16}$. calcolare l'errore rispetto al valore limite della successione che è il numero di Nepero e $(\exp(1))$.



Calcolo della precisione di macchina

La precisione di macchina è definita come il più piccolo numero floating point e tale che: $1+e=e$, che corrisponde al risultato della funzione `sys.float_info.epsilon`, oppure si può calcolare con la funzione seguente:

```
def calcola_precisione_macchina():  
    epsilon = 1.0  
    while 1.0 + epsilon / 2.0 != 1.0:  
        epsilon /= 2.0  
    return epsilon  
  
# Esegui il programma  
precisione = calcola_precisione_macchina()  
print("Precisione di macchina:", precisione)
```